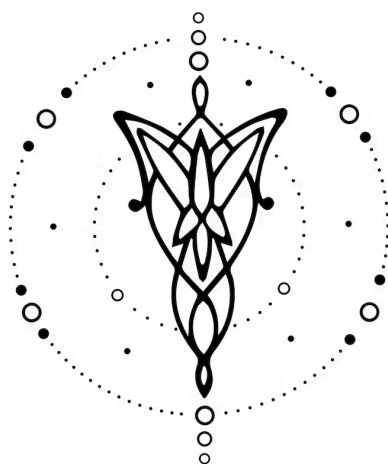




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Lautaro	Paletta	64499	Lautaro Paletta
Agustin	Ronda	64507	AGUSTIN LEONEL RONDA
Nicole	Salama	64488	NICOLE Yael SALAMA
Maria	Otegui	61204	MARIA OTEGUI

2. Repositorio

La solución y su documentación serán versionadas en: [Colorito](#).

3. Dominio

Desarrollar un lenguaje que permita realizar alteraciones visuales y generar derivados a partir de una o varias imágenes de entrada. El lenguaje debe permitir modificar aspectos de las imágenes como sus colores, tamaño, orientación, brillo y agregar extras como texto, filtros, pixelados o unir dos imágenes.

Para lograr esto, contaremos con instrucciones dentro de nuestro lenguaje de programación que nos permitirán aplicar estas transformaciones dada una o varias imágenes y parámetros de configuración para la transformación a aplicar. A partir de esto tendremos no solo un lenguaje de programación capaz de aplicar ciertas transformaciones, si no que las transformaciones variarán entre sí dependiendo de los parámetros que se le ingresen.

La implementación satisfactoria de este lenguaje permitiría reducir el tiempo que se pasa en los programas de diseño en tareas básicas, así como también dar la oportunidad de automatizar procesos que normalmente demoraría mucho tiempo a los usuarios que los realizan de forma repetitiva, sería cuestión de realizar un código preciso y ejecutarlo tantas veces como sea necesario.

4. Construcciones

El lenguaje desarrollado ofrecerá las siguientes funcionalidades:

- (I). Se podrán abrir imágenes especificando la ubicación en el dispositivo de la misma usando la instrucción *open*.
- (II). Se podrá recortar imágenes en partes cuya cantidad es potencias de dos usando la instrucción *crop*.
- (III). Se podrá redimensionar imágenes usando la instrucción *resize*.
- (IV). Se podrá rotar imágenes usando la instrucción *rotate*.
- (V). Se podrá reflejar imágenes usando la instrucción *flip*.
- (VI). Se podrá modificar la opacidad de imágenes usando la instrucción *opacity*.
- (VII). Se podrán mezclar dos imágenes con una determinada opacidad usando la instrucción *blend*.
- (VIII). Se podrán unir dos imágenes vertical u horizontalmente usando la instrucción *merge*.
- (IX). Se podrá convertir imágenes a una escala de grises usando la instrucción *greyscale*.
- (X). Se podrá invertir los colores de una imagen usando la instrucción *invert*.
- (XI). Se podrá ajustar el brillo de una imagen usando la instrucción *brightness*.
- (XII). Se podrá ajustar el contraste de una imagen usando la instrucción *contrast*.
- (XIII). Se podrá desenfocar una imagen usando la instrucción *blur*.
- (XIV). Se podrá aumentar la nitidez de una imagen usando la instrucción *sharpen*.
- (XV). Se podrá pixelar una imagen usando la instrucción *pixelate*.
- (XVI). Se podrá cambiar un rango de colores de una imagen usando la instrucción *recolor*.
- (XVII). Se podrá guardar una imagen, indicando la ruta usando la instrucción *save*.
- (XVIII). Todas las funciones devuelven la imagen modificada, no modifican la que se les pase por parámetro.

- (XIX). Las variables podrán ser de tipo *image*, *string*, *integer*, *percentage*, *dimension* y *color*.
- (XX). Se podrá asignar un valor a una variable usando el operador `=`.
- (XXI). Se podrá asignar el valor de una variable a otra variable usando el operador `=`.
- (XXII). Se deberá finalizar cada línea usando `;`.
- (XXIII). Todas las funciones que devuelvan un tipo *image* podrán anidarse entre sí usando `()` para delimitar su comienzo y final.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que corta una imagen en una potencia de 2.
- (II). Un programa que redimensiona el tamaño de una imagen.
- (III). Un programa que rote la imagen en la cantidad de grados que desee.
- (IV). Un programa que refleje la imagen horizontal o verticalmente.
- (V). Un programa que convierta una imagen en blanco y negro.
- (VI). Un programa que invierta los colores de una imagen en su negativo.
- (VII). Un programa que agregue texto sobre una imagen.
- (VIII). Un programa que abra una imagen.
- (IX). Un programa que aplique todas las operaciones disponibles en cuántas imágenes se abran en el programa logrando una imagen como resultado final.
- (X). Un programa que une dos imágenes horizontal o verticalmente.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado.
- (II). Un programa que no indique la ruta de la(s) imagen(es) que se quiere(n) utilizar.
- (III). Un programa que intente aplicar filtros que no existen en el lenguaje.
- (IV). Un programa que intente recortar una imagen en potencia de 2 partes.
- (V). Un programa que indique una ruta inexistente de una imagen.

6. Ejemplos

Obtener un recorte del primer cuarto de una imagen.

```
/* Abrir la imagen principal */
fotoPerfil = open "/imagenes/perfil.jpg";

save (crop fotoPerfil in 4 get 1) "./foto_perfil.jpg";
```

Redimensionar y rotar una imagen

```
/* Abre la imagen y aplica resize and rotate */
img = open “/img/retrato.jpg”;

save (rotate (resize img to 300x300) by 180) “./img.jpg”;
```

Mezclar dos imágenes con opacidad

```
/* Mezcla dos imágenes usando 50% de opacidad
Usa blend con 2 imágenes y guarda el resultado */
img1 = open “/img/background.jpg”;
img2 = open “/img/logo.jpg”;
result = blend img1 with img2 using 50%;
save result “out/background_logo.jpg”;
```

Bajar opacidad a una imagen

```
/* Bajar opacidad a una imagen */
img1 = open “/img/portada.jpg”;
img1_with_opacity = opacity img1 50%;
save img1_with_opacity “out/img1_op.jpg”;
```